



DC – AI – Routing – Switching – Security – Wireless – Automation

Packets Lost in Transit?

An HPE Juniper Networking Techpost

Anton Elita

2024-01-08

HPE



Contents

1	Packets Lost in Transit?	4
1.1	Packet Counters	4
1.2	Packet Sampling	4
1.3	Inline Monitoring	4
1.3.1	Junos Configuration	5
1.3.2	PCAP! Or didn't happen!	6
1.3.3	Exported Packet Types	7
1.3.4	Data Template	7
1.3.5	Options Template	9
1.3.6	Data Record	10
1.4	Collecting and Processing Inline Monitoring Data	13
1.4.1	On-Box Collector	13
1.4.2	Off-Box Collectors	17
1.5	Summary	17
1.6	Useful links	17
1.7	Glossary	18

List of Figures

1 Exported Packet 7

List of Tables

2 Fields and Common Property IDs 9

1 Packets Lost in Transit?

Anton Elita - 01/08/2024

Inline Monitoring allows to get forwarding status codes for transit packets, as well as a sample of the packet itself – even if the packet was dropped for whatever reason. By having the packet header exposed to a remote collector, this feature enables other nice applications like peering traffic visibility on a shared port, traffic anomaly detection and network performance observation.

1.1 Packet Counters

Most of the experienced readers would know where to look for packet counters, so we'll list only some of the widely known commands. Even if packet counters don't always allow to understand the exact reason if a drop (and don't give us the headers of the packet either), they are a good start of a troubleshooting process:

```
1 show interfaces extensive
2 show interfaces queue
3 show pfe statistics traffic
4 show interfaces voq # <- only for VOQ-based platforms like PTX
```

1.2 Packet Sampling

There are a few other ways to get a packet mirrored for further analysis: plain packet mirroring, sflow, or flow tap. They differ in support for packet types (layer-2, layer-3) and sampling rates, and granularity of the packet matching. Still, they don't deliver metadata such as exception details, and packets dropped at early stages of processing would not be reported.

1.3 Inline Monitoring

Characteristic	MX Series	PTX10k Series
Supported Sampling Rate	1:1 to 1:16M	1:1 to 1:32M
Format	IPFIX	IPFIX
Number of Instances	16	7
Collectors per Instance	4 collectors per instance. Independent sampling rate per collector, and per-collector stats	4 collectors per instance.Sampling rate per instance
Traffic Direction	Ingress, Egress	Ingress, Egress
Exported clip size	64-126 bytes, L2 included	64-256 bytes, L2 included (not for exception traffic in egress direction)
Attachment Points	Logical Interface, Filter	Logical Interface, Filter
Supported families	inet, inet6, mpls, vpls, bridge, ccc, any	inet, inet6, mpls, ethernet-switch, ccc, any

Characteristic	MX Series	PTX10k Series
Metadata	Interface indexes, direction, timestamp, flow byte / packet count, flow identification, exception reports	Interface indexes, direction, timestamp, flow byte / packet count, flow identification, exception reports
Implementation	Fully Inline	Fully Inline
Number of flows	Does not maintain flow state	Does not maintain flow state
Junos Release	19.4R1	22.4R1

1.3.1 Junos Configuration

- configuration for exporting a clip of exception packets, they will be part of “data records”
- template configuration with definitions of data records fields, they are part of “data template record”
- various options are reported as “options template record”
- an instance of inline monitoring with configurable keys
- collector definitions
- an example filter with packet matches. Adjust it before activating on any interface, so that the filter matches necessary traffic only
- interface with that filter applied on family any, direction ingress or egress. It is deactivated, as we do not intend to capture valid transit packets, though it is perfectly possible if you activate it.

Reporting exception packets does not require a firewall filter applied to an interface, the configuration is per PFE

Notes:

- a collector needs to capture both data template and data packets to understand the flexible format of the IPFIX record
- like in any traffic sampling scenario, using a firewall filter matching all packets is not a good idea – unless you know what you are doing
- if a filter is applied on egress direction, make sure you don’t sample the IPFIX packets themselves – exclude them in the filter!

```

1 aelita@pe2# show groups INLINE-MON
2 # PFE exception export
3 chassis {
4     fpc 0 {
5         pfe 0 {
6             exception-reporting {
7                 category packet-errors {
8                     inline-monitoring-instance in-monitor1;
9                 }
10                category layer3 {
11                    inline-monitoring-instance in-monitor1;
12                }
13                category layer2 {
14                    inline-monitoring-instance in-monitor1;
15                }
16                category forwarding-state {
17                    inline-monitoring-instance in-monitor1;
18                }
19                category firewall {
20                    inline-monitoring-instance in-monitor1;

```

```

21     }
22   }
23 }
24 }
25 }
26 # Data and options template
27 services {
28   inline-monitoring {
29     template {
30       template-1 {
31         template-refresh-rate 10;
32         option-template-refresh-rate 10;
33         primary-data-record-fields {
34           cpid-egress-interface-index;
35           cpid-forwarding-class-drop-priority;
36           cpid-forwarding-exception-code;
37           cpid-forwarding-nexthop-id;
38           cpid-ingress-interface-index;
39           cpid-underlying-ingress-interface-index;
40           datalink-frame-size;
41           direction;
42           egress-interface-snmp-id;
43           ingress-interface-snmp-id;
44         }
45       }
46     }
47     instance {
48       in-monitor1 {
49         template-name template-1;
50         maximum-clip-length 126;
51         collector {
52           aelita-vm {
53             source-address 1.0.0.6;
54             destination-address 11.20.0.2;
55             dscp 7;
56             destination-port 4739;
57             forwarding-class network-control;
58             sampling-rate 10;
59           }
60         }
61       }
62     }
63     observation-cloud-id 2;
64   }
65 }
66 # Optional filter applied to an interface/direction
67 inactive: interfaces { # deliberately leaving this deactivated
68   ge-0/0/0 {
69     unit 0 {
70       filter {
71         input inmon;
72       }
73     }
74   }
75 }
76 firewall {
77   family any {
78     filter inmon {
79       term A { # an example only, will catch all traffic
80         inline-monitoring-instance in-monitor1;
81         accept;
82       }
83     }
84   }
85 }
86 }

```

1.3.2 PCAP! Or didn't happen!

```
1 tcpdump -ni eth1 -s 9000 port 4739 -w /tmp/out.pcap
```

When tcpdump was stopped (e.g. with Ctrl-C), we can use wireshark or tshark to decode the contents of the packet dump.

```
1 > tshark -r /tmp/out.pcap -Y ip.src==1.0.0.6
```

```

2  1 0.000000 1.0.0.6 -> 11.20.0.2 CFLOW 134 IPFIX flow ( 92 bytes) Obs-Domain-ID=33619968 [Data
   -Template:384]
3  2 0.000001 1.0.0.6 -> 11.20.0.2 CFLOW 78 IPFIX flow ( 36 bytes) Obs-Domain-ID=33619968 [
   Options-Template:640]
4  3 0.000001 1.0.0.6 -> 11.20.0.2 CFLOW 70 IPFIX flow ( 28 bytes) Obs-Domain-ID=33619968 [Data
   :640]
5  4 10.009818 1.0.0.6 -> 11.20.0.2 CFLOW 134 IPFIX flow ( 92 bytes) Obs-Domain-ID=33619968 [
   Data-Template:384]
6  5 10.009818 1.0.0.6 -> 11.20.0.2 CFLOW 78 IPFIX flow ( 36 bytes) Obs-Domain-ID=33619968 [
   Options-Template:640]
7  6 10.009819 1.0.0.6 -> 11.20.0.2 CFLOW 70 IPFIX flow ( 28 bytes) Obs-Domain-ID=33619968 [Data
   :640]
8  [...]

```

All right, we have got three types of frames, but what are they exactly? Let's see in the next section.

1.3.3 Exported Packet Types

[RFC7011](#) defines three packet types used by IPFIX: *data template record*, *options template record* and *data record*. Why would we need all three? Well, at least for the automated decoding – when we know the templates, we can dissect the data records themselves.

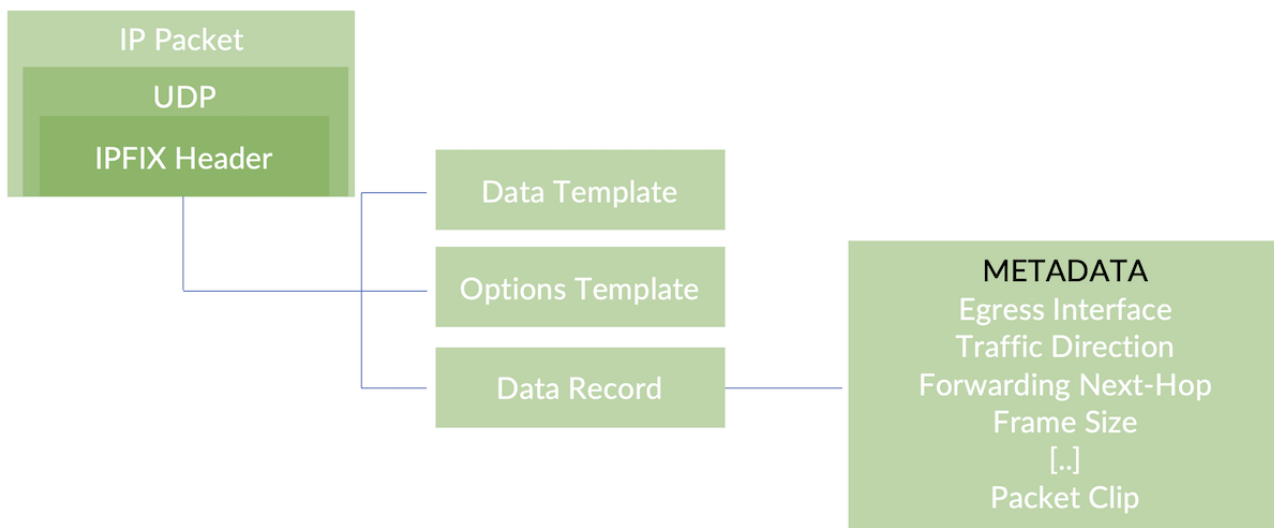


Figure 1: Exported Packet

If you have [tshark](#), using -V flag will help look deeper into the decoded data.

```
1 tshark -r /tmp/out.pcap -Y ip.src==1.0.0.6 -V
```

Here are the relevant outputs from the above shell command.

1.3.4 Data Template

```

1 Data template describes the fields carried by the data records:
2 [...] NetFlow/IPFIX
3   Version: 10
4   Length: 92
5   Timestamp: Dec 27, 2023 14:27:57.000000000 UTC
6   ExportTime: 1703687277
7   FlowSequence: 0 (expected 9)
8   [Expert Info (Warning/Sequence): Unexpected flow sequence for domain ID 33619968 (expected
   9, got 0)]
9   [Unexpected flow sequence for domain ID 33619968 (expected 9, got 0)]
10  [Severity level: Warning]

```

```

11      [Group: Sequence]
12      Observation Domain Id: 33619968
13      Set 1 [id=2] (Data Template): 384
14      FlowSet Id: Data Template (V10 [IPFIX]) (2)
15      FlowSet Length: 76
16      Template (Id = 384, Count = 11)
17      Template Id: 384
18      Field Count: 11
19      Field (1/11): OBSERVATION_DOMAIN_LEVEL_JUNIPER_COMMON_PROPERTIES
20      1... .. = Pen provided: Yes
21      .000 0000 1000 1001 = Juniper Resiliency:
22      OBSERVATION_DOMAIN_LEVEL_JUNIPER_COMMON_PROPERTIES (137)
23      Length: 4
24      PEN: Juniper Networks, Inc. (2636)
25      Field (2/11): OBSERVATION_DOMAIN_LEVEL_JUNIPER_COMMON_PROPERTIES
26      1... .. = Pen provided: Yes
27      .000 0000 1000 1001 = Juniper Resiliency:
28      OBSERVATION_DOMAIN_LEVEL_JUNIPER_COMMON_PROPERTIES (137)
29      Length: 2
30      PEN: Juniper Networks, Inc. (2636)
31      Field (3/11): OBSERVATION_DOMAIN_LEVEL_JUNIPER_COMMON_PROPERTIES
32      1... .. = Pen provided: Yes
33      .000 0000 1000 1001 = Juniper Resiliency:
34      OBSERVATION_DOMAIN_LEVEL_JUNIPER_COMMON_PROPERTIES (137)
35      Length: 4
36      PEN: Juniper Networks, Inc. (2636)
37      Field (4/11): OBSERVATION_DOMAIN_LEVEL_JUNIPER_COMMON_PROPERTIES
38      1... .. = Pen provided: Yes
39      .000 0000 1000 1001 = Juniper Resiliency:
40      OBSERVATION_DOMAIN_LEVEL_JUNIPER_COMMON_PROPERTIES (137)
41      Length: 4
42      PEN: Juniper Networks, Inc. (2636)
43      Field (5/11): OBSERVATION_DOMAIN_LEVEL_JUNIPER_COMMON_PROPERTIES
44      1... .. = Pen provided: Yes
45      .000 0000 1000 1001 = Juniper Resiliency:
46      OBSERVATION_DOMAIN_LEVEL_JUNIPER_COMMON_PROPERTIES (137)
47      Length: 4
48      PEN: Juniper Networks, Inc. (2636)
49      Field (6/11): OBSERVATION_DOMAIN_LEVEL_JUNIPER_COMMON_PROPERTIES
50      1... .. = Pen provided: Yes
51      .000 0000 1000 1001 = Juniper Resiliency:
52      OBSERVATION_DOMAIN_LEVEL_JUNIPER_COMMON_PROPERTIES (137)
53      Length: 4
54      PEN: Juniper Networks, Inc. (2636)
55      Field (7/11): INPUT_SNMP
56      0... .. = Pen provided: No
57      .000 0000 0000 1010 = Type: INPUT_SNMP (10)
58      Length: 4
59      Field (8/11): OUTPUT_SNMP
60      0... .. = Pen provided: No
61      .000 0000 0000 1110 = Type: OUTPUT_SNMP (14)
62      Length: 4
63      Field (9/11): DIRECTION
64      0... .. = Pen provided: No
65      .000 0000 0011 1101 = Type: DIRECTION (61)
66      Length: 1
67      Field (10/11): dataLinkFrameSize
68      0... .. = Pen provided: No
69      .000 0001 0011 1000 = Type: dataLinkFrameSize (312)
70      Length: 2
71      Field (11/11): dataLinkFrameSection
72      0... .. = Pen provided: No
73      .000 0001 0011 1011 = Type: dataLinkFrameSection (315)
74      Length: 65535 [i.e.: "Variable Length"]
75      [Expected Sequence Number: 9]
76      [Previous Frame in Sequence: 3]

```

Output 1: Data Template

Observation Domain ID is an identifier of the observation point, for example referring to a chassis. In this case it's automatically generated.

Fields 1 to 6 have the [Element ID](#) of 137 (Common Properties ID), associated with Juniper's Private Enterprise Number 2636 [allocated by IANA](#). The left-most 6 bits are encoding a common property ID (CP-ID) from Juniper, followed by reserved 0 bits. That's why when looking at the hex dump, one would see 0x4, which is a value of 1 in the left-most 6 bits.

The remaining fields 7 to 11 have well-known Element ID related to the purpose of the respective field.

The fields can be better described in a table.

Table 2: Fields and Common Property IDs

Field ID	Element ID	First byte value(last two bits are 0)	Notes
1	137	0x4	Forwarding Class
2	137	0x8	Exception Code
3	137	0xc	Forwarding Next-Hop ID [1]
4	137	0x10	Egress Logical Interface Index [2]
5	137	0x14	Ingress Interface ID (when part of LAG) [1]
6	137	0x18	Ingress Logical Interface Index [2]
7	10	n/a	Input SNMP Interface Index
8	14	n/a	Output SNMP Interface Index
9	61	n/a	Traffic Direction (Ingress = 0, Egress = 1)
10	312	n/a	Data Link Frame Size
11	315	n/a	Data Link Frame Section (packet clip)

[1] not supported on PTX

[2] Junos interface index (not SNMP ifIndex)

1.3.5 Options Template

```

1 [...] NetFlow/IPFIX
2   Version: 10
3   Length: 36
4   Timestamp: Dec 27, 2023 14:27:47.000000000 UTC
5   ExportTime: 1703687267
6   FlowSequence: 8 (expected 0)
7   [Expert Info (Warning/Sequence): Unexpected flow sequence for domain ID 33619968 (expected
8     0, got 8)]
9     [Unexpected flow sequence for domain ID 33619968 (expected 0, got 8)]
10    [Severity level: Warning]
11    [Group: Sequence]
12    Observation Domain Id: 33619968
13    Set 1 [id=3] (Options Template): 640
14    FlowSet Id: Options Template (V10 [IPFIX]) (3)

```

```

14     FlowSet Length: 20
15     Options Template (Id = 640) (Scope Count = 1; Data Count = 1)
16         Template Id: 640
17         Total Field Count: 2
18         Scope Field Count: 1
19         Field (1/1) [Scope]: FLOW_EXPORTER
20             0... .... = Pen provided: No
21             .000 0000 1001 0000 = Type: FLOW_EXPORTER (144)
22             Length: 4
23         Field (1/1): SAMPLING_INTERVAL
24             0... .... = Pen provided: No
25             .000 0000 0010 0010 = Type: SAMPLING_INTERVAL (34)
26             Length: 4
27         Padding: 0000
28     [Expected Sequence Number: 0]
29     [Previous Frame in Sequence: 1]

```

Output 2: Options Template

Flow Exporter and Sampling Interval are part of the options template.

1.3.6 Data Record

Finally, let us check the data received. Below is just the frame containing the data record. You will see that a relatively modern tshark (I used ubuntu 22.04 LTS) can very well understand both *IPFIX headers* as well as decode the *payload*!

```

1  [...] NetFlow/IPFIX
2  Version: 10
3  Length: 148
4  Timestamp: Dec 27, 2023 14:55:19.000000000 UTC
5  ExportTime: 1703688919
6  FlowSequence: 60 (expected 174)
7  [Expert Info (Warning/Sequence): Unexpected flow sequence for domain ID 33619968 (expected
8  174, got 60)]
9  [Unexpected flow sequence for domain ID 33619968 (expected 174, got 60)]
10 [Severity level: Warning]
11 [Group: Sequence]
12 Observation Domain Id: 33619968
13 Set 1 [id=384] (1 flows)
14     FlowSet Id: (Data) (384)
15     FlowSet Length: 132
16     [Template Frame: 1]
17     Flow 1
18         Observation Cloud Level Juniper Common Properties: Forwarding Class and Drop Priority
19         0000 01.. .... = Juniper CPID Type: Forwarding Class and
20         Drop Priority (0x01)
21         .... ..00 0000 0000 0000 0000 0000 0000 = Juniper CPID Value: 0 [Forwarding class: 0
22         Drop Priority: 0]
23         Observation Cloud Level Juniper Common Properties: Forwarding Exception Details
24         0000 10.. .... = Juniper CPID Type: Forwarding Exception Details (0x02)
25         .... ..00 1100 0010 = Juniper CPID Value: 194
26         Observation Cloud Level Juniper Common Properties: Forwarding Nexthop Details
27         0000 11.. .... = Juniper CPID Type: Forwarding Nexthop
28         Details (0x03)
29         .... ..00 0000 0000 0000 0000 0000 0000 = Juniper CPID Value: 0
30         Observation Cloud Level Juniper Common Properties: Egress Interface Details
31         0001 00.. .... = Juniper CPID Type: Egress Interface
32         Details (0x04)
33         .... ..00 0000 0000 0000 0000 0000 0000 = Juniper CPID Value: 0
34         Observation Cloud Level Juniper Common Properties: Ingress Underlying Interface Details
35         0001 01.. .... = Juniper CPID Type: Ingress Underlying
36         Interface Details (0x05)
37         .... ..00 0000 0000 0000 0000 0000 0000 = Juniper CPID Value: 0
38         Observation Cloud Level Juniper Common Properties: Ingress Interface Details
39         0001 10.. .... = Juniper CPID Type: Ingress Interface
40         Details (0x06)
41         .... ..00 0000 0000 0000 0001 0110 1000 = Juniper CPID Value: 360
42     InputInt: 576
43     OutputInt: 0
44     Direction: Ingress (0)
45     Data Link Frame Size: 94
46     Data Link Frame Section: 56041900370
47     e56041900b76e8847f42411ff4600004cd35100000111a4490a0024017f00...

```

```

40 Ethernet II, Src: 56:04:19:00:b7:6e (56:04:19:00:b7:6e), Dst: 56:04:19:00:37:0e
    (56:04:19:00:37:0e)
41 Destination: 56:04:19:00:37:0e (56:04:19:00:37:0e)
42 Address: 56:04:19:00:37:0e (56:04:19:00:37:0e)
43 .... ..1. .... = LG bit: Locally administered address (this is
    NOT the factory default)
44 .... ..0. .... = IG bit: Individual address (unicast)
45 Source: 56:04:19:00:b7:6e (56:04:19:00:b7:6e)
46 Address: 56:04:19:00:b7:6e (56:04:19:00:b7:6e)
47 .... ..1. .... = LG bit: Locally administered address (this is
    NOT the factory default)
48 .... ..0. .... = IG bit: Individual address (unicast)
49 Type: MPLS label switched packet (0x8847)
50 MultiProtocol Label Switching Header, Label: 1000001, Exp: 0, S: 1, TTL: 255
51 1111 0100 0010 0100 0001 .... = MPLS Label: 1000001 (0xf4241)
52 .... ..00. .... = MPLS Experimental Bits: 0
53 .... ..1. .... = MPLS Bottom Of Label Stack: 1
54 .... 1111 1111 = MPLS TTL: 255
55 Internet Protocol Version 4, Src: 10.0.36.1, Dst: 127.0.0.1
56 0100 .... = Version: 4
57 .... 0110 = Header Length: 24 bytes (6)
58 Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
59 0000 00.. = Differentiated Services Codepoint: Default (0)
60 .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
61 Total Length: 76
62 Identification: 0xd351 (54097)
63 Flags: 0x00
64 0... .... = Reserved bit: Not set
65 .0.. .... = Don't fragment: Not set
66 ..0. .... = More fragments: Not set
67 ...0 0000 0000 0000 = Fragment Offset: 0
68 Time to Live: 1
69 [Expert Info (Note/Sequence): "Time To Live" only 1]
70 ["Time To Live" only 1]
71 [Severity level: Note]
72 [Group: Sequence]
73 Protocol: UDP (17)
74 Header Checksum: 0xa449 [validation disabled]
75 [Header checksum status: Unverified]
76 Source Address: 10.0.36.1
77 Destination Address: 127.0.0.1
78 Options: (4 bytes), Router Alert
79 IP Option - Router Alert (4 bytes): Router shall examine packet (0)
80 Type: 148
81 1... .... = Copy on fragmentation: Yes
82 .00. .... = Class: Control (0)
83 ...1 0100 = Number: Router Alert (20)
84 Length: 4
85 Router Alert: Router shall examine packet (0)
86 User Datagram Protocol, Src Port: 49547, Dst Port: 3503
87 Source Port: 49547
88 Destination Port: 3503
89 Length: 52
90 Checksum: 0x1862 [unverified]
91 [Checksum Status: Unverified]
92 [Stream index: 3]
93 [Timestamps]
94 [Time since first frame: 0.000000000 seconds]
95 [Time since previous frame: 0.000000000 seconds]
96 UDP payload (44 bytes)
97 String_len_short: 94
98 [Expected Sequence Number: 174]
99 [Previous Frame in Sequence: 13]
100 Multiprotocol Label Switching Echo
101 Version: 1
102 Global Flags: 0x0001
103 0000 0000 0000 0... = Reserved: 0x0000
104 .... ..1. .... = Validate FEC Stack: True
105 .... ..0. .... = Respond only if TTL expired: False
106 .... ..0.. .... = Validate Reverse Path: False
107 Message Type: MPLS Echo Request (1)
108 Reply Mode: Reply via an IPv4/IPv6 UDP packet (2)
109 Return Code: No return code (0)
110 Return Subcode: 0
111 Sender's Handle: 0xa6e6c57f
112 Sequence Number: 5
113 Timestamp Sent: Dec 27, 2023 14:55:20.248490999 UTC
114 Timestamp Received: (0) Jan 1, 1970 00:00:00.000000000 UTC
115 Target FEC Stack
116 Type: Target FEC Stack (1)
117 Length: 8

```

```

118     FEC Element 1: Nil FEC
119     Type: Nil FEC (16)
120     Length: 4
121     Label 1: 0 (IPv4 Explicit-Null)

```

Output 3: Data Record and Packet Decode with tshark

That's a lot!

From the *IPFIX* header we see following characteristics of the packet:

- Best-Effort class of service (value 0)
- drop priority 0
- it's an exception packet, exception id 194 (see below how to decode this value)
- forwarding next-hop, egress interface, OutputInt index are all equal to 0

it's another confirmation that the packet is an exception packet - Input Interface ID is 360, and InputInt SNMP index is 576:

```

1 aelita@pe2# run show interfaces | match 576
2   Logical interface ge-0/0/0.0 (Index 360) (SNMP ifIndex 576)

```

Now we know the interface details and some other metadata of the transit packet and can proceed to analyze the *packet* itself (using same output above coming from tshark packet decoder):

- Ethernet Frame, Destination and Source MACs, Protocol ID 0x8847 (MPLS)
- MPLS label stack [1000001]
- IPv4 payload, SRC/DST IP, TTL, Router Alert option
- UDP payload, SRC/DST ports
- MPLS echo request, sequence number, NIL FEC

That is a great insight, we know that this packet caused an exception, and even the exact content of the packet!

Why was there an exception created? We need to understand what the exception code 194 means.

In Junos, on MX series of routers, there are counters associated with exceptions of type discard and punt. There are many reasons for discard, for example dropping packets where the route to destination does not exist (or explicitly points to the discard interface), VLAN mismatch, wrong destination MAC, bad IP headers and many other reasons and counters associated with them. A punted packet is sent to the upper layers for further processing, for example control traffic directed to the router itself, or when there is a need to resolve next-hop like ARP, or various subscriber packets like PPPoE – again, counters associated with those and other similar actions.

There are ongoing efforts to get forwarding status codes standardized, for example [draft-mvmd-opsawg-ipfix-fwd-exceptions](#) at IETF.

Values for status codes used by MX series of routers can be found by using PFE level commands:

```

1 aelita@pe2> request pfe execute target fpc0 command "show jnh 0 exceptions terse"
2 SENT: Ukern command: show jnh 0 exceptions terse
3
4
5 Reason Type Packets Bytes
6 =====
7
8
9 PFE State Invalid
10 -----

```

```

11 unknown family DISC( 73) 42 3002
12
13
14 Packet Exceptions
15 -----
16 IP options PUNT( 2) 2 64
17
18
19 Routing
20 -----
21 discard route DISC( 66) 29230 2046300
22 hold route DISC( 70) 4377 482852
23 resolve route PUNT( 33) 16805 1494766
24 control pkt punt via nh PUNT( 34) 36482 5778200
25 host route PUNT( 32) 55951 4855476

```

Output 4: Exceptions List

The list is short as it shows only non-zero counters. For the full list, remove “terse”. The “type” column shows the type (discard or punt) and the forwarding status code. Status codes reported via IPFIX have an offset of +128, so that the reported status code 194 is equal to 66 in the PFE output above (194-128=66), i.e., a discard route.

A quick validation on the label 1000001 in the FIB confirms it will be discarded:

```

1 aelita@pe2# run show route forwarding-table label 1000001 extensive table default
2 Routing table: default.mpls [Index 0]
3 MPLS:
4
5 Destination: default
6 Route type: permanent
7 Route reference: 0 Route interface-index: 0
8 Multicast RPF nh index: 0
9 P2mpidx: 0
10 Flags: sent to PFE
11 Next-hop type: discard Index: 50 Reference: 2
12
13 Routing table: default.l2xc [Index 0]
14 L2XC:
15
16 Destination: default
17 Route type: permanent
18 Route reference: 0 Route interface-index: 0
19 Multicast RPF nh index: 0
20 P2mpidx: 0
21 Flags: none
22 Next-hop type: discard Index: 50 Reference: 2

```

Output 5: FIB Validation

```

1 aelita@p3> ping mpls segment-routing spring-te label-stack labels [ 1000001 ] nexthop-interface
   ge-0/0/2.0 nexthop-address 10.0.36.2

```

1.4 Collecting and Processing Inline Monitoring Data

1.4.1 On-Box Collector

For a quick debugging, on-box collector can be used for storing forwarding exceptions:

```

1 aelita@pe2# show | compare
2 [edit groups INLINE-MON]
3 + system {
4 +   resiliency {
5 +     exceptions {
6 +       forwarding;
7 +     }
8 +   store {
9 +     fwding-file jri-fwd.log;
10 +   }

```

```

11 + }
12 + }
13 [edit groups INLINE-MON services inline-monitoring instance in-monitor1 collector]
14     aelita-vm { ... }
15 + localhost {
16 +   source-address 1.0.0.6;
17 +   destination-address 1.0.0.6;
18 +   destination-port 4739;
19 +   sampling-rate 10;
20 + }

```

Output 6: On-Box Collector

The above is a patch to the previous configuration – you can paste it using “load patch terminal” in the configuration mode. This is somewhat less known (but very useful!) way to load the diff directly.

Please note that on-box collector configuration must specify a locally configured “destination-address” and the port must be 4739.

The log file (in CLI: “show log jri-fwd.log”) will quickly be populated with the same three distinct types of IPFIX reports.

Options Template:

```

1 ***** Netflow/IPFIX *****
2 Version: 10
3 Length: 28
4 Export time: 1703754885
5 Seq no: 68
6 ObservationDomainID: 33620480
7 setID: 640
8 setLength: 12
9 Options Data
10 fieldType: exportingProcessId, field_length: 4
11 value: 1
12 fieldType: samplingInterval, field_length: 4
13 value: 10

```

Data Template:

```

1 ***** Netflow/IPFIX *****
2 Version: 10
3 Length: 92
4 Export time: 1703754895
5 Seq no: 35
6 ObservationDomainID: 33620480
7 setID: 2
8 setLength: 76
9 DataTemplate
10 templateID: 384
11 fieldCount: 11
12 fieldType: commonPropertiesID, fieldLength: 4, enterpriseNumber: 2636
13 fieldType: commonPropertiesID, fieldLength: 2, enterpriseNumber: 2636
14 fieldType: commonPropertiesID, fieldLength: 4, enterpriseNumber: 2636
15 fieldType: commonPropertiesID, fieldLength: 4, enterpriseNumber: 2636
16 fieldType: commonPropertiesID, fieldLength: 4, enterpriseNumber: 2636
17 fieldType: commonPropertiesID, fieldLength: 4, enterpriseNumber: 2636
18 fieldType: ingressInterface, fieldLength: 4
19 fieldType: egressInterface, fieldLength: 4
20 fieldType: flowDirection, fieldLength: 1
21 fieldType: dataLinkFrameSize, fieldLength: 2
22 fieldType: dataLinkFrameSection, fieldLength: 65535

```

Data Report:

```

1 ***** Netflow/IPFIX *****
2 Version: 10
3 Length: 148
4 Export time: 1703756312
5 Seq no: 115
6 ObservationDomainID: 33620480
7 setID: 384
8 setLength: 132

```

```

 9  field_type: forwardingClass, field_length: 4
10  value: 0
11  field_type: exceptionCode, field_length: 2
12  value: Discard Route
13  field_type: nhIndex, field_length: 4
14  value: 0
15  field_type: oifIndex, field_length: 4
16  value: 0
17  field_type: underlyingiifIndex, field_length: 4
18  value: 0
19  field_type: iifIndex, field_length: 4
20  value: 360
21  field_type: ingressInterface, field_length: 4
22  value: 576
23  field_type: egressInterface, field_length: 4
24  value: 0
25  field_type: flowDirection, field_length: 1
26  value: 00
27  field_type: dataLinkFrameSize, field_length: 2
28  value: 94
29  field_type: dataLinkFrameSection, field_length: 94
30  value: 56041900 370e5604 1900b76e 8847f424
31  11ff4600 004c4844 00000111 2f570a00
32  24017f00 00019404 0000d7b1 0daf0034
33  686f0001 00010102 0000a254 21010000
34  0001e937 c098d84c be1e0000 00000000
35  00000001 00080010 00040000 0000

```

Output 7: On-Box Collected Data

Among other metadata like interface details, we see that the reported packet took a “Discard Route”: the forwarding status code of 194 has automatically been decoded for human eyes.

However, the packet itself is represented as a sequence of hex bytes, and needs further decoding for a better understanding.

There are a few online decoders available, but why not do that in a shell of e.g. a linux-based computer like this:

```
1 cat <<EOF | xxd -r -p | od -Ax -tx1 | text2pcap - - | tshark -r - -V
```

Then just paste the hex code followed by EOF on an empty line and see the results. The magic happens by converting hex bytes to a hex dump (function of xxd and od), generating a pcap from the hex dump (text2pcap) and a verbose decoding of the pcap (tshark).

Complete output would look like this:

```

1 cat <<EOF | xxd -r -p | od -Ax -tx1 | text2pcap - - | tshark -r - -V
2 pipe pipe pipe pipe heredoc> 56041900 370e5604 1900b76e 8847f424
3 11ff4600 004c4844 00000111 2f570a00
4 24017f00 00019404 0000d7b1 0daf0034
5 686f0001 00010102 0000a254 21010000
6 0001e937 c098d84c be1e0000 00000000
7 00000001 00080010 00040000 0000
8 EOF
9 Input from: Standard input
10 Output to: Standard output
11 Output format: pcapng
12
13 -----
14 Read 1 potential packet, wrote 1 packet (384 bytes including overhead).
15 Frame 1: 94 bytes on wire (752 bits), 94 bytes captured (752 bits) on interface Fake IF,
    text2pcap, id 0
16   Section number: 1
17   Interface id: 0 (Fake IF, text2pcap)
18   Interface name: Fake IF, text2pcap
19   Encapsulation type: Ethernet (1)
20   Arrival Time: Dec 28, 2023 11:28:38.000001000 CET
21   [Time shift for this packet: 0.000000000 seconds]
22   Epoch Time: 1703759318.000001000 seconds
23   [Time delta from previous captured frame: 0.000000000 seconds]
24   [Time delta from previous displayed frame: 0.000000000 seconds]
25   [Time since reference or first frame: 0.000000000 seconds]

```

```

26   Frame Number: 1
27   Frame Length: 94 bytes (752 bits)
28   Capture Length: 94 bytes (752 bits)
29   [Frame is marked: False]
30   [Frame is ignored: False]
31   [Protocols in frame: eth:ethertype:mpls:ip:udp:mpls-echo]
32   Ethernet II, Src: 56:04:19:00:b7:6e (56:04:19:00:b7:6e), Dst: 56:04:19:00:37:0e
    (56:04:19:00:37:0e)
33   Destination: 56:04:19:00:37:0e (56:04:19:00:37:0e)
34   Address: 56:04:19:00:37:0e (56:04:19:00:37:0e)
35   .... 01. .... = LG bit: Locally administered address (this is NOT the
        factory default)
36   .... 00. .... = IG bit: Individual address (unicast)
37   Source: 56:04:19:00:b7:6e (56:04:19:00:b7:6e)
38   Address: 56:04:19:00:b7:6e (56:04:19:00:b7:6e)
39   .... 01. .... = LG bit: Locally administered address (this is NOT the
        factory default)
40   .... 00. .... = IG bit: Individual address (unicast)
41   Type: MPLS label switched packet (0x8847)
42   MultiProtocol Label Switching Header, Label: 1000001, Exp: 0, S: 1, TTL: 255
43   1111 0100 0010 0100 0001 .... = MPLS Label: 1000001 (0xf4241)
44   .... 0000. .... = MPLS Experimental Bits: 0
45   .... 0001. .... = MPLS Bottom Of Label Stack: 1
46   .... 1111 1111 = MPLS TTL: 255
47   Internet Protocol Version 4, Src: 10.0.36.1, Dst: 127.0.0.1
48   0100 .... = Version: 4
49   .... 0110 = Header Length: 24 bytes (6)
50   Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
51   0000 00.. = Differentiated Services Codepoint: Default (0)
52   .... 0000 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
53   Total Length: 76
54   Identification: 0x4844 (18500)
55   000. .... = Flags: 0x0
56   0... .... = Reserved bit: Not set
57   .0... .... = Don't fragment: Not set
58   ..0. .... = More fragments: Not set
59   ...0 0000 0000 0000 = Fragment Offset: 0
60   Time to Live: 1
61   [Expert Info (Note/Sequence): "Time To Live" only 1]
62   ["Time To Live" only 1]
63   [Severity level: Note]
64   [Group: Sequence]
65   Protocol: UDP (17)
66   Header Checksum: 0x2f57 [validation disabled]
67   [Header checksum status: Unverified]
68   Source Address: 10.0.36.1
69   Destination Address: 127.0.0.1
70   Options: (4 bytes), Router Alert
71   IP Option - Router Alert (4 bytes): Router shall examine packet (0)
72   Type: 148
73   1... .... = Copy on fragmentation: Yes
74   .00. .... = Class: Control (0)
75   ...1 0100 = Number: Router Alert (20)
76   Length: 4
77   Router Alert: Router shall examine packet (0)
78   User Datagram Protocol, Src Port: 55217, Dst Port: 3503
79   Source Port: 55217
80   Destination Port: 3503
81   Length: 52
82   Checksum: 0x686f [unverified]
83   [Checksum Status: Unverified]
84   [Stream index: 0]
85   [Timestamps]
86   [Time since first frame: 0.000000000 seconds]
87   [Time since previous frame: 0.000000000 seconds]
88   UDP payload (44 bytes)
89   Multiprotocol Label Switching Echo
90   Version: 1
91   Global Flags: 0x0001
92   0000 0000 0000 0... = Reserved: 0x0000
93   .... 0001 = Validate FEC Stack: True
94   .... 0000 = Respond only if TTL expired: False
95   .... 0000 = Validate Reverse Path: False
96   Message Type: MPLS Echo Request (1)
97   Reply Mode: Reply via an IPv4/IPv6 UDP packet (2)
98   Return Code: No return code (0)
99   Return Subcode: 0
100  Sender's Handle: 0xa2542101
101  Sequence Number: 1
102  Timestamp Sent: Dec 28, 2023 09:38:32.844920999 UTC
103  Timestamp Received: Jan 1, 1970 00:00:00.000000000 UTC

```

```

104 Target FEC Stack
105   Type: Target FEC Stack (1)
106   Length: 8
107   FEC Element 1: Nil FEC
108     Type: Nil FEC (16)
109     Length: 4
110     Label 1: 0 (IPv4 Explicit-Null)

```

Output 8: Packet Decode

1.4.2 Off-Box Collectors

For the off-box collection if IPFIX with metadata, [goflow2](#) or [pmacct](#) can be used. I will use goflow2 as an example here – just to give an idea how an off-box collector could be implemented.

```

1 # docker run -p 4739:2055/udp -ti netsampler/goflow2:latest
2 INFO[0000] Starting GoFlow2
3 INFO[0000] Starting collection count=1 hostname= port=2055 scheme=netflow
4 INFO[0000] Starting collection count=1 hostname= port=6343 scheme=sflow
5 {"Type":"IPFIX","TimeReceived":1703760968,"SequenceNum":3924,"SamplingRate":10,"FlowDirection":0,"SamplerAddress":"1.0.0.6","TimeFlowStart":0,"TimeFlowEnd":0,"TimeFlowStartMs":0,"TimeFlowEndMs":0,"Bytes":94,"Packets":1,"SrcAddr":"10.0.36.1","DstAddr":"127.0.0.1","Etype":2048,"Proto":17,"SrcPort":37892,"DstPort":0,"InIf":576,"OutIf":0,"SrcMac":"56:04:19:00:b7:6e","DstMac":"56:04:19:00:37:0e","SrcVlan":0,"DstVlan":0,"VlanId":0,"IngressVrfId":0,"EgressVrfId":0,"IpTos":0,"ForwardingStatus":0,"IpTtl":1,"TcpFlags":0,"IcmpType":0,"IcmpCode":0,"Ipv6FlowLabel":0,"FragmentId":46935,"FragmentOffset":0,"BiFlowDirection":0,"SrcAs":0,"DstAs":0,"NextHop":"","NextHopAs":0,"SrcNet":0,"DstNet":0,"BgpNextHop":[],"BgpCommunities":[],"AsPath":[],"HasMpls":true,"MplsCount":1,"Mpls_1Ttl":255,"Mpls_1Label":1000001,"Mpls_2Ttl":0,"Mpls_2Label":0,"Mpls_3Ttl":0,"Mpls_3Label":0,"MplsLastTtl":0,"MplsLastLabel":0,"MplsLabelIp":[],"ObservationDomainId":33619968,"ObservationPointId":0,"CustomInteger_1":0,"CustomInteger_2":0,"CustomInteger_3":0,"CustomInteger_4":0,"CustomInteger_5":0,"CustomBytes_1":[],"CustomBytes_2":[],"CustomBytes_3":[],"CustomBytes_4":[],"CustomBytes_5":[]}

```

Right out of the box, goflow2 can already recognize many fields. [Customization](#) of the default mapping is required for other non-standard fields.

1.5 Summary

The whole telecommunications industry is moving fast towards reliable and robust automated networks. Being able to quickly determine any malfunctioning component is a key element in that journey.

Juniper MX and PTX devices can use their ASIC capabilities to generate inline monitoring of the transit traffic, without adverse effects on the line-card or central processing unit. IPFIX format is ASIC-friendly, and flexible to accommodate ever growing list of exported elements.

1.6 Useful links

- Monitoring, Sampling, and Collection Services Interfaces User Guide: <https://www.juniper.net/documentation/us/en/software/junos/flow-monitoring/topics/topic-map/resiliency-exception-reporting.html>
- [TechPost] Detection of Blackholes Using Juniper Resiliency Interface: <https://juniper.github.io/techposts/detection-of-blackholes-in-networks-using-jri/article>
- [TechPost] Sampling Evolution: <https://juniper.github.io/techposts/sampling-evolution/article>
- TCPdump: <https://www.tcpdump.org/>
- [RFC] Specification of the IP Flow Information Export (IPFIX) Protocol for the Exchange of Flow Information: <https://www.rfc-editor.org/rfc/rfc7011>

- tshark: <https://tshark.dev/>
- [IANA] IP Flow Information Export (IPFIX) Entities: <https://www.iana.org/assignments/ipfix/ipfix.xhtml>
- [IANA] PRIVATE ENTERPRISE NUMBERS: <https://www.iana.org/assignments/enterprise-numbers/enterprise-numbers>
- [IETF Draft] IP Flow Information Export (IPFIX) Information Elements Extension for Forwarding Exceptions: <https://datatracker.ietf.org/doc/html/draft-mvmd-opsawg-ipfix-fwd-exceptions-08#name-forwardingstatuscode>
- goflow2: <https://github.com/netsampler/goflow2>
- netsampler / goflow2 mapping: <https://github.com/netsampler/goflow2/blob/main/cmd/goflow2/mapping.yaml>
- PMACCT: <http://www.pmacct.net/>

1.7 Glossary

- ARP: Address Resolution Protocol
- ASIC: Application Specific Integrated Circuits
- CLI: Common Line Interface
- CP-ID: Common Property Identity
- FIB: Forwarding Information Base
- IETF: Internet Engineering Task Force
- IP: Internet Protocol
- IPFIX: IP Flow Information Export
- JRI: Juniper Resiliency Interface
- LAG: Ling Aggregation
- MAC: Media Access Control
- MPLS: Multiprotocol Label Switching
- NIL FEC: Nil Forward Error Correction
- PFE: Packet Forwarding Engine
- PPPoE: Point-to-Point Protocol over Ethernet
- PCAP: Packet Capture
- SNMP: Simple Network Management Protocol
- VLAN: Virtual Local Area Network
- VOQ: Virtual Output Queue



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.

Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.

Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com

